

# Present Value Engine Code Walkthrough

A maintainer's guide to the present-value worksheet: field-presence solving across lump-sum, periodic and rate blocks, COLA escalation, variable-rate discounting, and the underlying geometric-series mathematics.

---

**Scope.** The PV engine (PRESVALU.pas), the variable-rate “fancy” engine and table output (PVLXSCRN.pas, pvltable.pas, PVLUTIL.pas), the screen layer (PresentValueScreenUnit.pas), and the shared math (INTSUTIL.pas).

**Audience.** Engineers maintaining or porting the financial logic. Every claim is anchored to a file:line reference.

**Authority.** The DOS source is authoritative. Line numbers cite legacy/src/dos\_source/; the annotated mirror legacy/src\_documented/dos\_source/ carries inline commentary (shifted line numbers).

**Actuarial.** The life-contingency (“for life” / pay-on-death) integration that weaves through this engine is covered in the companion actuarial\_dos\_walkthrough.pdf; here it is noted only where it hooks in.

# Contents

---

## 0 How to read this document

## 1 Architecture & data model

1.1 The three blocks · 1.2 Records & status codes · 1.3 Rates, COLA & sentinels

## 2 The UI layer: five grids

2.1 Grids & containers · 2.2 The rate three-views · 2.3 COLA & Simple/Compound · 2.4 The calc trigger & the plain↔fancy toggle

## 3 Entry & classification

3.1 Enter · 3.2 FirstPass · 3.3 The field-presence → solve-path map

## 4 The summation mathematics

4.1 SumFormula · 4.2 Summation · 4.3 COLA & stepped escalation

## 5 Forward calc

## 6 Backward calc

## 7 Variable-rate (“fancy”) present value

## 8 Table output

## 9 A worked trace & maintainer notes

## 10 Appendix: call graph & file:line index

## 0 How to read this document

The present-value screen is a worksheet that values streams of money across time. It has three editable blocks: **lump-sum** single payments, **periodic** payment streams (with optional cost-of-living escalation), and a **rate / present-value** block holding the discount rate and the as-of date. As with every Per%Sense screen, the user leaves one cell blank and the engine solves for it — an amount, a date, a rate, or the total value. A “fancy” mode replaces the single discount rate with a *schedule* of rates over time.

### Citation convention

References look like `PRESVALU.pas:1105`; the number is where the construct begins. **All line numbers cite** `legacy/src/dos_source/`. The annotated mirror `legacy/src_documented/dos_source/` has the same code with inline comments (shifted lines).

### The cast of files

| File                       | Role                                                                                                                                                                     |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PRESVALU.pas               | Engine. Enter dispatcher, FirstPass classifier, FrontwardCalc (forward + rate/as-of solves), BackwardCalc (amount/date solves), and the Summation geometric-series math. |
| PVLXSCRN.pas               | Variable-rate (“fancy”) engine: FancyFirstPass, FancySummation, COLA escalation.                                                                                         |
| PVLUTIL.pas                | ValueOfOnePayment — the single-payment discount primitive that walks a rate schedule.                                                                                    |
| pvltable.pas               | MakePVLTable — merges all series by date and emits the detail + summary schedule.                                                                                        |
| PresentValueScreenUnit.pas | UI. Five grids, the rate three-views, COLA conversion, the calc trigger.                                                                                                 |
| INTSUTIL.pas               | Shared math (exxp/lnn, rate conversions, YearsDif) — see amortization walkthrough §5.                                                                                    |

**The one formula to know.** The present value of a periodic stream is  $\text{amtn} \times \text{Summation}$ , and Summation is a geometric series  $(1-f^n)/(1-f)$  whose ratio  $f = e^{(\text{cola}-\text{rate})/P}$  nets escalation against discounting. When  $\text{cola} = \text{rate}$ ,  $f \rightarrow 1$  and the series flattens to  $n$  — the single most important edge case in the whole engine.

# 1 Architecture & data model

## 1.1 The three blocks

The worksheet is three arrays of rows plus a rate block, all discounted to a common as-of date:

```
BLOCK 1  a[]  lump-sum single payments      (date, amount, value)
BLOCK 2  b[]  periodic payment streams      (from, to, per-yr, amount, COLA, value)
BLOCK 3  c[]  rate / present-value lines    (as-of, rate, sumvalue, duration)

fancy mode adds:  cc[]  rate-line schedule (date, rate)      d  footer (xasof, simple, xvalue)
```

Each row solves the field the user left blank (the “2-of-3” rule for lumps: any two of date/amount/value give the third). sumvalue in BLOCK 3 is the total present value the screen reports and the backward solver targets.

## 1.2 Records & status codes

Per-field inout status codes `PETYPES.PAS:112–142`: empty=0 (blank, the solve request), outp=1 (computed; also the provisional fromcalc flag), defp=2 (known/derived), inp=3 (typed), badp=-1 (flagged bad). The guard status  $\geq$  defp means “known”. A per-row aggregate *line-status* code `PETYPES.PAS:365–376` summarizes a row: fully\_specified=254 (forward-computable), contains\_unknown=253 (one blank to solve), over\_determined=255, missing\_N=250–252.

| Record                          | Key fields                                                                                           | Source               |
|---------------------------------|------------------------------------------------------------------------------------------------------|----------------------|
| lumpsum (a)                     | date, amt0 (face amount at its date), val0 (PV at as-of), act0 (contingency code)                    | PETYPES.PAS:566      |
| periodic (b)                    | fromdate/todate, peryr, amtn, cola, valn (PV of stream), ninstallments, actn                         | PETYPES.PAS:582      |
| raterec                         | rate (the stored True/continuous rate), peryr (its compounding), status (fromrate/fromapr/fromyield) | PETYPES.PAS:600      |
| presval (c)                     | asof, r:raterec, sumvalue, duration                                                                  | PETYPES.PAS:610      |
| rateline (cc) /<br>xpresval (d) | fancy rate band (date, r); fancy footer (xasof, simple, xvalue)                                      | PETYPES.PAS:622, 630 |

## 1.3 Rates, COLA & sentinels

The stored rate is a **continuously-compounded** rate; cola is a continuous annual escalation rate.

df.c.colamonth selects the COLA policy: 1..12 = escalate on that month each year; CNT=98 = continuous; ANN=99 = anniversary-of-start. Key cutoffs: teeny= $10^{-10}$  (zero-rate boundary), tiny= $10^{-5}$  (Taylor-series band), small= $10^{-4}$ . The sentinel year latest.y = 249 marks a stream that runs “forever” (a perpetuity).

## 2 The UI layer: five grids

### 2.1 Grids & containers

The form hosts five grids, each backed by a global record array `PresentValueScreenUnit.pas:40–65, 210–275`:

| Grid             | Container | Role                                                          |
|------------------|-----------|---------------------------------------------------------------|
| LumpSumGrid      | a[]       | single payments (date / amount / value)                       |
| PeriodicGrid     | b[]       | periodic streams (from / to / per-yr / amount / COLA / value) |
| PresentValueGrid | c[]       | plain single-rate PV (as-of / True / Loan / Yield / value)    |
| RatelineGrid     | cc[]      | variable-rate bands (date / True / Loan / Yield)              |
| XPresValGrid     | d         | fancy footer (as-of / computation / value)                    |

The grid→recordmarshallers (`AssignLumpSumValues` :1019, `AssignPeriodicValues` :1120, etc.) follow the universal blank-vs-typed rule: `Value='' → status := empty` (the field-presence signal); otherwise parse and set inp. Lump amount and value are mutually exclusive (entering one clears the other, :1044); typing any periodic column auto-fills peryr from the default :1096.

### 2.2 The rate three-views

The rate block shows one rate three ways — **True** (effective per-period), **Loan APR**, and **Yield** — and editing any one recomputes the others. The stored `r.rate` is always the True (continuous) rate

`PresentValueScreenUnit.pas:1271–1332`:

```
enter True (status fromrate): Loan = YieldFromRate(rate, peryr), Yield = YieldFromRate(rate, 1)
enter Loan (status fromapr): rate = RateFromYield(value, peryr)
enter Yield (status fromyield): rate = RateFromYield(value, 1)
```

When the payments-per-year changes, `FixRates` :2229 re-derives every `fromapr` row so the displayed APR tracks the new compounding.

### 2.3 COLA & Simple/Compound

COLA is entered as a percent and stored as a fraction (/100, :1175; ×100 on display). The fancy footer's **Computation** combo sets `d.simple`: in Simple mode the three rate views all show the raw number (no compounding conversion); in Compound mode they show the converted views :1613–1670.

### 2.4 The calc trigger & the plain↔fancy toggle

Enter in any cell calls `DoCalculation` `PresentValueScreenUnit.pas:313–327`, which loads the used-row counts into `nlines[]`, then — **in fancy mode** — pins the first rate band to the epoch (`cc[1].date := earliest`; `datestatus := defp`, :321) so the earliest payment is always covered, and calls the engine's `Enter(no_tab)`. Results flow back via `AllPresentValueData2Grids`. `ToggleAdvanced` :334 flips the global `pvl.fancy`, swapping block geometry so BLOCK 3 becomes the rate-line block `cc`.

## 3 Entry & classification

### 3.1 Enter

Enter(code) `PRESVALU.pas:1105–1281` runs `FirstPass`, warns before overwriting any already-determined BLOCK-3 line, re-classifies, then dispatches: `frontward` and `backward` → “too many unknowns”; `backward` → `BackwardCalc`. After a backward solve it forward-computes the remaining rate lines top-down (for `i := nlines` downto `stopat` do `FrontwardCalc(i)`, `:1253`; downto keeps line 1's solved result on screen). Actuarial entry points (`PrepareForLife`, `ActuarialCalc`) hook in here — see the actuarial walkthrough.

### 3.2 FirstPass

`FirstPass` `PRESVALU.pas:544–654` classifies each block (delegating to `FancyFirstPass` when `pvlfancy`). It counts present fields to assign each row a line-status code, deletes empty rows, then resolves two flags: `frontward` is true only if some row is `fully_specified` and *no* row still contains an unknown `:641–653`; `backward` is set inside the line-value computations when a `contains_unknown` row coexists with a fully-specified rate line.

### 3.3 The field-presence → solve-path map

The blank field selects the path (labels match the Go port's `BackwardKind`). Note PV-8/PV-9 solve a *BLOCK-3* cell (forward side, in `FrontwardCalc`); PV-1/2/4/5/6 solve a *payment* cell (backward side, in `BackwardCalc`). There is no PV-3 or PV-7.

| Blank field                        | Block | Path | Lines              |
|------------------------------------|-------|------|--------------------|
| lump amount (date+value given)     | 1     | PV-1 | PRESVALU:866–891   |
| lump date (amount+value given)     | 1     | PV-2 | PRESVALU:892–931   |
| periodic amount (both dates+value) | 2     | PV-4 | PRESVALU:943–956   |
| periodic to-date (from+amt+value)  | 2     | PV-5 | PRESVALU:965–999   |
| periodic from-date (to+amt+value)  | 2     | PV-6 | PRESVALU:1000–1070 |
| BLOCK-3 rate (payments + sumvalue) | 3     | PV-8 | PRESVALU:693–754   |
| BLOCK-3 as-of date                 | 3     | PV-9 | PRESVALU:755–818   |

## 4 The summation mathematics

The PV of one periodic stream is  $\text{amtn} \times \text{Summation}$ . Throughout:  $r$  = discount rate,  $c$  = COLA,  $P$  =  $\text{RealPerYr}(\text{peryr})$  (real payments/yr),  $n$  =  $\text{ninstallments}$ , and  $\text{YearsDif}(\text{asof}, \text{date})$  is negative for future payments (so the exponential discounts).

### 4.1 SumFormula — the geometric kernel

`SumFormula(lnf, n)` [PRESVALU.pas:248–267](#) is the closed form of  $\sum_{i=0}^{n-1} e^{i \cdot \text{lnf}}$ :

$$\text{SumFormula} = (1 - e^{n \cdot \text{lnf}}) / (1 - e^{\text{lnf}}), \quad \rightarrow n \text{ when } |\text{lnf}| < \text{teeny}$$

The **teeny cutoff** (per-period growth  $\approx 0$ ) degenerates the series to  $n$  equal terms. A **tiny**-band branch swaps in 2nd-order Taylor expansions of  $1 - e^x$  so the  $0/0$ -shaped ratio stays accurate near zero.

### 4.2 Summation

`Summation(k, j)` [PRESVALU.pas:365–464](#) sets the per-period net log-growth  $\text{lnf} := (\text{cola} - r)/P$  — escalation working against discounting — and (in the common closed-form path) returns:

$$\text{Summation} = e^{r \cdot \text{since}} \cdot (1 - f^n) / (1 - f) \cdot [\text{phase factor}], \quad f = e^{\text{lnf}} = e^{(c-r)/P}$$

where `since` measures time from the as-of date to the stream's reference point. The branch picks whether to measure from a “standard loan date” one period before the first payment (when  $\text{asof} \leq \text{fromdate}$ ) or from the to-date, with a phase factor ( $e^{-r/P}$ ) correcting the one-period offset [:451–457](#).

**The cola = rate degeneracy.** When  $c = r$ ,  $\text{lnf} = 0$ ,  $f = 1$ , and the geometric ratio  $(1 - f^n) / (1 - f)$  is  $0/0$ . The teeny branch handles it by collapsing to  $n$  equal terms: every escalated payment discounts back to exactly the same present value, so the stream is worth  $n$  times one payment's PV. Every backward solver special-cases this (PV-5/PV-6 use `AddNPeriods(..., round(val/amtn))`) to dodge the division by zero.

Two other paths exist: a **perpetuity guard** (if  $\text{COLA} \geq \text{rate}$  and the stream runs forever, the value is infinite — flagged badp, [:379–383](#)) and an **exact** period-by-period sum used for weekly/biweekly, life-contingent, or explicitly-exact modes [:389–410](#).

### 4.3 COLA & stepped escalation

Continuous COLA ( $\text{colamonth} = \text{CNT}$ ) is baked into  $\text{lnf}$ . Anniversary COLA ( $\text{colamonth} \in 1..12$  or  $\text{ANN}$ ) is a *discrete step* — `SummationForSteppedCola` [PRESVALU.pas:269–363](#) keeps the payment constant within a COLA year and multiplies by  $e^{\text{cola}}$  at each anniversary  $\text{coladate}$ . Its fast path splits time into a first partial period, a whole-year middle block summed with two `SumFormula` calls, and a final partial period.

## 5 Forward calc

FrontwardCalc(k) PRESVALU.pas:656–826 operates on rate line k with three mutually-exclusive unknowns.

### Unknown total value (the plain forward PV) :669–692

$$\text{sumvalue} = \sum_{\text{lumps}} \text{amt}0 \cdot e^{r \cdot \text{YearsDif}(\text{asof}, \text{date})} + \sum_{\text{periodics}} \text{amtn} \cdot \text{Summation}$$

Each lump is compounded/discounted from its date to the as-of date; each stream is its amount times the summation factor. (Actuarial weighting and the POD term fold in here — see the actuarial walkthrough.)

### Unknown rate (PV-8) — secant iteration :693–754

First guess  $r.\text{rate} := 0.1$ ; recompute the sum at the trial rate; step by a numerical-derivative secant  $\text{diff} := (\text{sumvalue} - \text{sum}) \cdot \text{diff} / (\text{sum} - \text{oldsum})$ , clamped to  $\pm 0.04$ . Up to 30 iterations; on the first failure it restarts from rate 0, then reports “Rate computation did not converge.” A guard at iteration 2 detects a zero step — meaning the sum doesn’t depend on rate — and tells the user to specify amounts, not values :725–730.

### Unknown as-of date (PV-9) — fixed-point :755–818

Present value scales as  $\text{sum} \cdot e^{r \cdot \Delta \text{asof}}$ , so setting it equal to the target gives the step directly:

$$\Delta \text{asof} = \ln(\text{sumvalue} / \text{sum}) / r \rightarrow \text{AddYears}(\text{asof}, \Delta \text{asof})$$

Iterated (because sum depends weakly on asof) until  $|\Delta| < 0.002$  years ( $< 1$  day); it “should always converge on the first pass.” Refuses when  $|r| < \text{teeny}$  (“interest rate too small”).



## 6 Backward calc

BackwardCalc PRESVALU.pas:828–1085 first **derives the target**: start from the screen total  $val := c[1].sumvalue$ , then subtract every already-known row's contribution ( $val -= val0 / val -= valn$ , :852–862). What remains is the present value the single unknown row must supply. (Under life-contingency the POD value is peeled off first — see the actuarial walkthrough.)

| Path | Solve              | Method                                                                                                                          | Lines     |
|------|--------------------|---------------------------------------------------------------------------------------------------------------------------------|-----------|
| PV-1 | lump amount        | direct: $amt0 = val \cdot e^{-r \cdot \text{YearsDif}(\text{date}, \text{asof})}$ (compound the PV forward to the payment date) | 869–891   |
| PV-2 | lump date          | Newton on $amt0 \cdot e^{r \cdot \text{yrs}} = val0$ ; $\leq 30$ iters, derivative $amt0 \cdot r \cdot e^{r \cdot \text{yrs}}$  | 892–931   |
| PV-4 | periodic amount    | direct: $amtn = val / \text{Summation}$                                                                                         | 943–956   |
| PV-5 | periodic to-date   | closed-form first guess, then $\pm 1$ -period snap to a true installment boundary                                               | 965–999   |
| PV-6 | periodic from-date | two analytic approximations + $\pm 1$ -period snap                                                                              | 1000–1070 |

The date solves (PV-5/PV-6) follow a consistent pattern: an analytic approximation places the date near the answer, then a discrete refinement loop steps one period at a time until the result stops improving (snapping to an integral installment count), and finally an **amount correction** :1071–1078 scales  $amtn$  by  $targetval / valn$  to absorb the residual from forcing a whole-period count.

**Boundary cases the solvers guard.** Sign consistency (a positive amount cannot have negative value, :899); the  $cola=$ rate degeneracy ( $\text{AddNPeriods}(\dots, \text{round}(val/amtn))$  instead of a log, :969); perpetuities (finite only if  $r > cola$ ); and non-convergence reporting after the iteration caps.

## 7 Variable-rate (“fancy”) present value

Plain PV discounts every cash flow with one rate. Fancy PV replaces it with the `cc[] rate schedule` — an ascending list of (date, rate) bands partitioning the timeline. To move a dollar between its payment date and the as-of date, the engine walks the bands and multiplies a *piecewise product* of per-band factors.

### 7.1 `ValueOfOnePayment` — the discount primitive

`ValueOfOnePayment(howmuch, when)` `PVLUTIL.pas:25–66` values one payment at `d^.xasof` through the schedule:

$$\text{value} = \text{howmuch} \cdot \prod_{\text{bands } b} e^{\pm \text{rate}_b \cdot \Delta \text{years}_b}$$

The sign is + to compound a past payment forward to the as-of date, – to discount a future one back. A single rate would be one factor over the whole span; the *product over bands* is exactly what makes a schedule differ. (A Simple-interest variant accrues additively instead.)

### 7.2 `FancySummation` & the fancy passes

`FancySummation(j)` `PVLXSCRN.pas:305–352` sums a periodic stream installment-by-installment, each term `ValueOfOnePayment(colamt, movingdate)` with COLA applied per period by `UpdateAmountWithCola` `:103–113` (continuous  $e^{\text{cola} \cdot \Delta t}$  or stepped  $\times e^{\text{cola}}$  at each anniversary). It is scaled to a unit payment, so the same number drives both forward (`valn = amtn · sum`) and backward (`amtn = valn / sum`) solves. `FancyFirstPass` `:430–517` classifies the footer and rate bands (enforcing ascending dates), computes line values, and either solves backward or totals forward via `ComputeGrandTotal` `:420`.

**The “fast” fancy helpers are dead code.** `ValueOfPaymentSeries`, `ValueOfPastPayments`, and `ValueOfFuturePayments` `PVLXSCRN.pas:230, 261, 295` were a planned  $O(\text{bands})$  accelerated path but are stubs/incomplete; an `if (true)` or `... guard` `:316` forces the exact  $O(\text{installments})$  summation. The Go port intentionally does not port them. Likewise the backward-solve cell buffer (`FixPointers/Store/Recall`) is inert in fancy mode `PVLUTIL.pas:85` — fancy backward solves use the unit-scaled division instead.

## 8 Table output

---

MakePVTable `pvltable.pas:88–638` merges every lump and periodic series into one date-ordered stream and prints detail + cumulative rows. SortPayments `:211` sorts and seeds each series' cursor (capping perpetual streams at 50 years). SelectNextPayment `:272` is the merge heart: it finds the earliest pending date across all series, sums any co-dated payments, and applies COLA at emission time via the same UpdateAmountWithCola. TimeIsRipe `:331` decides when a cumulative subtotal is due (when a month in the user-chosen cumset has been passed). PrintNextPayment `:506` discounts the merged payment —  $\text{ValueOfOnePayment}(q, t)$  (fancy) or  $q \cdot e^{-r \cdot \text{YearsDif}(t, \text{asof})}$  (plain) — and accumulates the running total.

## 9 A worked trace & maintainer notes

**Trace.** One periodic stream: **\$1,000/month for 10 years, 5% rate, no COLA, value blank** (solve the value).

| # | What happens                                                                                                                                                              | Code                                         |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| 1 | Cells typed; value blank → <code>valnstatus := empty</code> . Rate stored as the True rate.                                                                               | <code>PresentValueScreenUnit.pas:1120</code> |
| 2 | Enter → <code>DoCalculation</code> → <code>Enter(no_tab)</code> .                                                                                                         | <code>PresentValueScreenUnit.pas:313</code>  |
| 3 | <code>FirstPass:ninstallments = NumberOfInstallments(from,to,12) = 120</code> ; row <code>fully_specified</code> (amount present, value blank) → <code>frontward</code> . | <code>PRESVALU.pas:607,644</code>            |
| 4 | <code>FrontwardCalc</code> unknown-value path: <code>valn = 1000 * Summation</code> ; $\ln f = (0 - 0.05) / 12$ , $f = e^{\ln f} < 1$ , so the geometric sum is finite.   | <code>PRESVALU.pas:682</code>                |
| 5 | <code>valnstatus := outp</code> ; <code>sumvalue</code> aggregates; results written back, value coloured as computed.                                                     | <code>PRESVALU.pas:692</code>                |

Now blank the **amount** and supply a value: step 4 becomes PV-4, `amtn = val / Summation :949`. Set `cola = 0.05` equal to the rate:  $\ln f = 0$ , the summation flattens to  $n = 120$ , and the stream is worth exactly 120 discounted payments — the degeneracy of \$4.2.

### Maintainer notes & known quirks

- **cola = rate is the load-bearing edge case.** The geometric ratio is  $0/0$ ; the teeny branch collapses it to  $n$ . Every forward and backward path special-cases it. Test it explicitly.
- **Rate stored as True (continuous); three views are derived.** Editing Loan/Yield round-trips through `RateFromYield/YieldFromRate`. A `peryr` change re-derives `fromapr` rows via `FixRates` `PresentValueScreenUnit.pas:2229`.
- **Perpetuities** (`todate.y = 249`) are finite only when `rate > cola`; otherwise the engine flags an infinite value `PRESVALU.pas:379`.
- **Fancy “fast” paths are dead code** `PVLXSCRN.pas:316`; only the exact installment summation runs. `cc[1]` is force-anchored to `earliest` and is not user-editable.
- **The `{#ifdef V_3} const_signal` block** in PV-6 `PRESVALU.pas:1003` is dead in the authoritative build (`V_3` never defined) and is intentionally not ported.
- **Two-pass FirstPass and the downto loop** in `Enter` are deliberate: the second pass re-derives state cleared by the overwrite warning, and `downto` preserves line 1's backward result.

## 10 Appendix: call graph & file:line index

```

Enter(code)                                PRESVALU.pas:1105
├─ FirstPass (classify; set frontward/backward) PRESVALU.pas:544
│   └─ ComputeLumpsumLineValues / ComputePeriodicLineValues :169 / :466
│       └─ (fancy) FancyFirstPass          PVLXSCRN.pas:430
├─ (backward) BackwardCalc                PRESVALU.pas:828
│   └─ derive target (subtract known val0/valn)
│       └─ PV-1 amount = val·e^{r·(date-asof)} :869
│       └─ PV-2 date   Newton                  :892
│       └─ PV-4 amount = val / Summation       :943
│       └─ PV-5 to-date / PV-6 from-date (approx + ±1-period snap) :965 / :1000
├─ (frontward) FrontwardCalc(i) downto stopat PRESVALU.pas:656
│   └─ unknown value  $\sum \text{amt0} \cdot e^{r \cdot \Delta} + \sum \text{amtn} \cdot \text{Summation}$  :669
│   └─ PV-8 rate      secant (numeric derivative) :693
│   └─ PV-9 asof      fixed-point  $\Delta = \ln(\text{SV}/\text{sum})/r$  :755
│   └─ Summation → SummationForSteppedCola → SumFormula :365 / :269 / :248
Variable-rate: FancySummation → ValueOfOnePayment (per band) PVLXSCRN:305 / PVLUTIL:25
Table:         MakeTable → MakePVLTable → SelectNextPayment / PrintNextPayment PRESVALU:1283 / pvltable:
UI:           DoCalculation → Enter(no_tab) PresentValueScreenUnit.pas:313

```

| Symbol                                              | Reference                                 |
|-----------------------------------------------------|-------------------------------------------|
| Records (lump / periodic / rate / presval)          | PETYPES.PAS:566 / 582 / 600 / 610         |
| Status & line-status codes                          | PETYPES.PAS:112–142 / 365–376             |
| Enter / FirstPass                                   | PRESVALU.pas:1105 / 544                   |
| FrontwardCalc / BackwardCalc                        | PRESVALU.pas:656 / 828                    |
| Summation / SumFormula / stepped COLA               | PRESVALU.pas:365 / 248 / 269              |
| FancySummation / FancyFirstPass                     | PVLXSCRN.pas:305 / 430                    |
| ValueOfOnePayment                                   | PVLUTIL.pas:25                            |
| MakePVLTable / SelectNextPayment / PrintNextPayment | pvltable.pas:88 / 272 / 506               |
| UI grids / calc trigger / FixRates                  | PresentValueScreenUnit.pas:40, 313 / 2229 |

Per%Sense DOS→Go port. Line numbers cite legacy/src/dos\_source/; the annotated mirror legacy/src\_documented/dos\_source/ carries inline commentary. The life-contingency integration that weaves through this engine is documented in actuarial\_dos\_walkthrough.pdf. Companion: amortization\_dos\_walkthrough.pdf, mortgage\_dos\_walkthrough.pdf.